NHẬP MÔN LẬP TRÌNH - ÔN TẬP CUỐI KÌ I

I. NỘI DUNG ÔN TẬP LÝ THUYẾT:

1. Mảng:

- **Khái niệm:**
  - Biểu diễn một dãy các phần tử có cùng kiểu và mỗi phần tử trong mảng biểu diễn 1 giá trị.
  - Kích thước mảng được xác định ngay khi khai báo và không thay đổi.
  - Một kiểu dữ liệu có cấu trúc do người lập trình định nghĩa.
  - Ngôn ngữ lập trình C luôn chỉ định một khối nhớ liên tục cho một biến kiểu mảng.

Ví dụ : dãy các số nguyên, dãy các kí tự

+ 1 2 3 4 5 6 7
+ a b c d e f

- **Các yếu tố xác định mảng**
  - Tên mảng
  - Kiểu dữ liệu
  - Số chiều
  - Kích thước

A/ MẢNG MỘT CHIỀU

1. <u>Khai báo</u>

Cú pháp:

<Kiểu dữ liệu> <Tên biến mảng>[<Số phần tử mảng>];

- Trong đó:

Kiểu dữ liệu: int, float, char

Tên biến mảng: 1 ký tự hoặc 1 dãy ký tự viết liền nhau và không có khoảng trắng

Số phần tử mảng: số lượng các phần tử của mảng 1 chiều

Ví dụ : int array[10000]; (khai báo một mảng kiểu số nguyên có độ dài là 10000)

char A[100]; (khai báo một mảng kiểu kí tự có độ dài là 100)

Có thể khai báo và liệt kê các giá trị của mảng:

<Kiểu dữ liệu> <tên mảng>[] = {danh sách};

- Ví dụ: int a[] = {0, 1, 2, 3, 4, 5};

Chúng ta đang khai báo 6 phần tử kiểu số nguyên, với các giá trị lần lượt là : a[0]=0, a[1]=1, a[2]...

Chú ý:

•Phải xác định cụ thể **<số phần tử mảng>** ngay lúc khai báo, không được sử dụng biến hoặc hằng thường.

Trường hợp muốn khai báo số phần tử là một biến thì biến đó phải được khai báo là hằng số

```
#include <iostream>
using namespace std;
const int N=100000;
int main () {
        int a[N];
        For (int i=
}
```

2. Chỉ số và cách truy xuất

- **Chỉ số:**

- Chỉ số mảng (vị trí trong mảng) là một giá trị số nguyên không âm.
- Chỉ số bắt đầu là 0 và không vượt quá số lượng phần tử tối đa -1 trong mảng.

int A[5];

0	1	2	3	4
99	17	50	43	72

Tên mảng: A

Kiểu dữ liệu của từng phần tử trong mảng: int

Số phần tử tối đa trong mảng: 5 phần tử

Các chỉ số được đánh số: 0 → 4 (0, 1, 2, 3, 4)

- **Truy xuất**

- Truy xuất phần tử mảng thông qua chỉ số:
  <Tên biến mảng>[<chỉ số mảng>]
- Các phần tử mảng là 1 dãy liên tục có chỉ số từ 0 đến <Số phần tử mảng>-1

int A[4]

0	1	2	3
29	137	50	4

Các truy xuất hợp lệ: A[0], A[1], A[2], A[3]

Các truy xuất không hợp lệ: A[-1], A[4], A[5]

Giá trị các phần tử mảng A[0]=29, A[1]=137, A[2]=50, A[3]=4

3. Cách nhập - xuất

- *Nhập mảng*

```
void nhapmang(int A[], int N)
{
        for(int i=0; i<N; i++){
                cout << "Nhap phan tu thu: " << i;
                cin >> A[i];
        }
}
```

- *Xuất mảng*

```
        void xuatmang(int A[], int N)
        {
                for(int i=0; i<N; i++)
                        cout << A[i];
        }
```

4. Một số ví dụ xử lí mảng

```
//Thuật toán sắp xếp
void swap(int &a, int &b)
{
  int temp= a;
  a = b;
  b = temp;
}
void exchangeSort(int a[], int n)
{
  for (int i = 0; i < n - 1; i++)
  for (int j = i + 1; j < n; j++)
  if (a[i] > a[j])
  swap(a[i], a[j]);
}
```

```
//Thuật toán đếm SL số nguyên tố trong mảng
bool check(int n) {
  if (n <= 1) return false;
      for (int i = 2; i <= sqrt(n); i++)
        if (n % i == 0)
              return false;
     return true;
}
int dem(int a[], int n) {
      int count = 0;
      for (int i = 0; i < n; i++) {
          if (check(a[i]) == true)
              count++;
      }
      return count;
}
```

```
//thuật toán chèn phần tử x vào vị trí k
của mảng
bool Insert(int *a, int &n, int x, int k)
{
  if (k<0 || k>= n) return false;
  for (int i = n-1; i >= k ; i--)
  {
  a[i + 1] = a[i];
  }
  a[k] = x;
  n++;
  return true;
}
```

```
//thuật toán tách phần tử thứ k của mảng
bool Remove(int *a, int &n, int k)
{
  if (k<0 || k>=n) return false;
  for (int i = k; i < --n; i++)
  {
  a[i] = a[i + 1];
  }
  n--;
  return true;
}
```

B/ MẢNG HAI CHIỀU

<kiểu dữ liệu> <tên mảng>[<số hàng>][<số cột>] ;

- Ví dụ: int a[2018][2019];
  float b[10][10];

Các thao tác hoàn toàn giống mảng 1 chiều, chỉ khác nếu ở mảng 1 chiều chỉ cần sử dụng 1 vòng lặp để duyệt thì mảng 2 chiều sẽ sử dụng 2 vòng lặp để duyệt.

2. Chuỗi:

C-style string (chuỗi trong C)

- Trong ngôn ngữ lập trình C, chuỗi là một tập hợp các ký tự (char) được lưu trữ trên các ô nhớ liên tiếp và luôn có ký tự `null` là `\0` báo hiệu kết thúc chuỗi.
- Ví dụ: khi khai báo và khởi tạo chuỗi theo cách trên, trình biên dịch sẽ tự thêm 1 ký tự null vào cuối.

```
char c[] = "c string";
```

c		s	t	r	i	n	g	\0

1.1 Cách khởi tạo

- Bạn có thể khởi tạo giá trị cho chuỗi theo 1 trong các cách dưới đây:

```
char c[] = "abcd";
char c[50] = "abcd";
char c[] = {'a', 'b', 'c', 'd', '\0'};
char c[5] = {'a', 'b', 'c', 'd', '\0'};
```

- Nhưng không được làm như sau:

```
char c[5] = "abcde";
```

Bởi vì chuỗi bạn có 5 ký tự cộng thêm 1 ký tự `\0` là 6, vượt quá kích thước.

1.2 Cách nhập xuất chuỗi:

- Có thể dung hàm scanf() để đọc không có khoảng trắng.
- Có thể dung hàm gets() để đọc chuỗi có khoảng trắng.
- Có thể dung puts() lẫn printf() để in chuỗi có khoảng trắng và chuỗi không có khoảng trắng. Tuy nhiên dung puts() sẽ tự động xuống hàng.
- Ví dụ:

```
#include <stdio.h>
int main()
{
    char name[20];
    printf("Enter name: ");
    scanf("%s", name);
    printf("Your name is %s.", name);
    return 0;
}
```

1.3 Các hàm cho C-style string.

- Cho 2 chuỗi s1, s2. Ta có các hàm thường dùng sau đây (phải include thư viện <string.h>):

+ strlen(s1): Tính toán độ dài chuỗi s1, không tính ký tự kết thúc (ký tự `\0`).
+ strcpy(s1,s2): Copy chuỗi s2 qua s1.

+ strcmp(s1,s2): So sánh hai string s1 và s2. Trả về 1 nếu s1>s2, 0 nếu s1=s2, -1 nếu s1<s2.
+ strcat(s1,s2): Nối chuỗi s2 vào cuối s1.

```
1  #include <stdio.h>
2
3  // Khai báo sử dụng thư viện string.h
4  #include <string.h>
5
6  int main() {
7
8     // Khai báo một String (String literal).
9     char s1[] = "This is ";
10
11    // Khai báo một string theo kiểu khai báo mảng
12    // Với phần tử null ở cuối.
13    char s2[] = { 't', 'e', 'x', 't', '\0' };
14
15    // Hàm: size_t strlen(const char *str)
16    // Sử dụng hàm strlen để kiểm tra độ dài chuỗi.
17    // size_t là kiểu dữ liệu số nguyên không dấu.
18    size_t len1 = strlen(s1);
19    size_t len2 = strlen(s2);
20
21    printf("Length of s1 = %d \n", len1);
22    printf("Length of s2 = %d \n", len2);
23
24    // Khai báo một mảng ký tự có 100 phần tử
25    // (100 phần tử là đủ dùng trong trường hợp này).
26    char mystr[100];
27
28    // Hàm: char *strcpy(char *dest, const char *src)
29    // Copy string s1 vào mystr
30    strcpy(mystr, s1);
31
32    // Hàm: char *strcat(char *dest, const char *src)
33    // Sử dụng hàm strcat để nối s2 vào mystr
34    strcat(mystr, s2);
35
36    // In ra nội dung của string.
37    printf("Your string = %s", mystr);
38
39    fflush(stdout);
40
41    return 0;
42 }
```

C++-style String (Chuỗi trong C++)

-C++ cung cấp cho bạn class string, nó giúp bạn làm việc dễ hàng hơn với các chuỗi. Các phương thức mà class string cung cấp vẫn hỗ trợ để làm việc với các C-style string.

-Tuy nhiên do chương trình bài học trên lớp nên mình sẽ không đi sâu vào phần này mà để cho các bạn tự đọc: https://o7planning.org/vi/10477/huong-dan-c-cpp-string

3. Cấu trúc:

a. Khái niệm: Struct (Kiểu cấu trúc) cho phép định nghĩa một vài loại biến có thể giữ giá trị của một vài thành viên cùng kiểu dữ liêu.

b. Định nghĩa (Khai báo) một cấu trúc trong C++:

- Để định nghĩa cấu trúc, ta phải sử dụng câu lệnh **struct**. Câu lệnh **struct** định nghĩa một kiểu dữ liệu mới, với hơn một thành viên trong chương trình của bạn. Dạng tổng quát của câu lệnh struct như sau đây:

```
struct <tên kiểu cấu trúc>
{
        <kiểu dữ liệu> <tên thành phần 1>;
        …
        <kiểu dữ liệu> <tên thành phần n>;
};
```

Ví dụ

```
struct SinhVien
{
      char mssv[10];
      char hoten[250];
      char lop[10];
};
```

c. Truy xuất:

- Sử dụng toán tử truy cập (.) để truy xuất. Toán tử truy cập thành viên cấu trúc được mã hóa là dấu chấm giữa tên biến cấu trúc và thành viên cấu trúc mà bạn muốn truy cập.
- Cú pháp: <tên biến cấu trúc>.<thuộc tính>

Ví dụ:

```
SinhVien p;
p.mssv = '19520205';
p.ten = 'Doan Ngoc Lam';
p.lop = 'KTPM2019';
```

d. Đặc điểm:

- Cấu trúc này có thể là thành phần của cấu trúc kia (có thể sử dụng lại)
- Struct có thể tự trỏ lại nó (đệ quy)
- Cần lưu ý về phạm vi sử dụng của kiểu dữ liệu tự định nghĩa cũng tương tự phạm vi sử dụng của biến trong chương trình, rnhưng việc định nghĩa kiểu dữ liệu không yêu cầu hệ điều hành cấp phát bộ nhớ nên hoàn toàn không làm ảnh hưởng đến tài nguyên của hệ thống. Do đó, chúng ta nên định nghĩa kiểu dữ liệu mới cho phạm vi toàn cục (global scope) để kiểu dữ liệu mới này có thể được sử dụng trong toàn bộ file, thậm chí là sử dụng trong những file mã nguồn khác trong cùng project.

4. Con trỏ:

a) Khái niệm: Một con trỏ là một biến, nó chứa địa chỉ vùng nhớ của một biến khác, chứ không lưu trữ giá trị của biến đó.

b) Tại sao con trỏ được dùng?

– Con trỏ có thể được sử dụng trong một số trường hợp sau:

+ Để trả về nhiều hơn một giá trị từ một hàm.

+ Thuận tiện hơn trong việc truyền các mảng và chuỗi từ một hàm đến một hàm khác.

+ Sử dụng con trỏ để làm việc với các phần tử của mảng thay vì truy xuất trực tiếp vào các phần tử này.

+ Để cấp phát bộ nhớ động và truy xuất vào vùng nhớ được cấp phát này.

+ Con trỏ còn có thể trỏ đến một biến Struct.

c) Khai báo con trỏ:

- Một con trỏ trước khi sử dụng cần được khai báo trước và có cú pháp như sau:

<kiểu con trỏ> *<tên con trỏ>;

d) Cách dùng

- Sử dụng con trỏ để trỏ đến địa chỉ của một biến.

+ Cú pháp: <tên con trỏ>= &<tên biến>;

- Sử dụng con trỏ để lấy hoặc cập nhập giá trị của một biến.

+ Cú pháp: *<tên con trỏ>=<tên biến>;

- Con trỏ và cấp phát động.

+ Cấp phát 1 phần tử: <tên con trỏ>= new <kiểu dữ liệu>;

+ Cấp phát nhiều phần tử: <tên con trỏ>= new <kiểu dữ liệu>[số phần tử];

+ Giải phóng bộ nhớ một biến: delete <tên con trỏ>;

+ Giải phóng bộ nhớ một mảng: delete[] <tên con trỏ>;

- Con trỏ với mảng:

```
int main()
{
        int a[10], *p;
        p=a;// => p=&a[0] => *(p+i)=*(a+i)=a[i]
}
```

5. Đệ quy:

a) Khái niệm: Một hàm được gọi là đệ quy nếu bên trong của hàm có lời gọi lại chính nó một cách tường minh hay bí ẩn.

b) Cấu trúc chung:

```
<kiểu trả về> <tên hàm>(danh sách tham số hình thức)
{
        if(điều kiện dừng)
        {
                dãy lệnh;
                return <trả về biểu thức>;
        }
        dãy lệnh;
}
```

c) Phân loại:

- Đệ quy tuyến tính: Một hàm được gọi là đệ quy tuyến tính khi bên trong thân hàm có duy nhất một lời gọi hàm gọi lại chính nó một cách tường minh.
- Đệ quy nhị phân: Một hàm được gọi là đệ quy nhị phân khi bên trong thân hàm của nó có đúng hai lời gọi hàm lại chính nó.
- Đệ quy hỗ tương: Một hàm được gọi là đệ quy hỗ tương khi bên trong thân hàm thứ nhất có lời gọi hàm tới hàm thứ 2 và bên trong thân hàm thứ 2 cũng có lời gọi hàm tới hàm thứ nhất.
- Đệ quy phi tuyến.

II. BÀI TẬP:

A. Giải đề thi:

https://pastebin.com/QmBKPFCG

B. Giải bài tập trọng tâm:

1. Mảng:

Phần 1: MẢNG MỘT CHIỀU

BT1: **Vòng tròn nam nữ**

Có M bạn nam và N bạn nữ được sắp xếp vào một vòng tròn

Yêu cầu: Tính mức chênh lệch giữa số cặp nam-nam (2 bạn nam đứng cạnh nhau) và nữ-nữ (hai bạn nữ đứng cạnh nhau)

Input

- Dòng đầu tiên 2 số nguyên dương M,N (0<=M,N<=106)
- Dòng tiếp theo là một dãy nhị phân biểu thị trạng thái của vòng tròn (1 là nam,0 là nữ) mỗi số cách nhau một khoảng trắng

Output

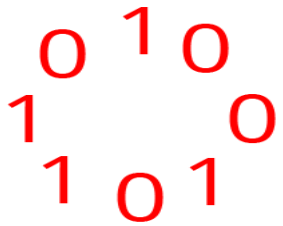
- Một dòng duy nhất là kết quả bài toán

Ví dụ

Input	Output
4 4 1 0 0 1 0 1 1 0	0

Giải thích

- Với M=4,N=4 và chuỗi trạng thái là 10010110, ta có vòng tròn như hình dưới



Có 1 cặp nam-nam (1-1) và 1 cặp nữ-nữ (0-0), vậy mức chênh lệch giữa cặp nam-nam và nữ-nữ là 0

Lời giải:

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int m,n;
    cin>>m>>n;

    int a[200000];
    for (int i=0;i<m+n;i++)
        cin>>a[i];
    int nam=0,nu=0;
    //biến đếm số cặp nam nữ đứng cạnh nhau
    a[m+n]=a[0];
    //xem như mảng đã thành vòng tròn
    for(int i=0;i<m+n;i++)
    {
        if(a[i]==a[i+1]&&a[i]==0) nu++;
        if(a[i]==a[i+1]&&a[i]==1) nam++;
    }
    cout<<abs(nam-nu);
}
```

BT2: **Dãy con tăng liên tiếp**

Mảng con (subarray) là một dãy liên tục các phần tử bên trong một mảng khác lớn hơn, vốn phân biệt với một dãy con (subsequence) là một dãy các phần tử không đòi hỏi phải liên tục ở bên trong một dãy khác.

Mảng con là một đề tài nghiên cứu thú vị về thuật toán với một số bài toán kinh điển và khó như: Mảng con tối đại, mảng con chung dài nhất, v.v... Trong khuôn khổ môn học này chúng ta sẽ đến với một dạng mảng con đơn giản hơn đó là *mảng con tăng*. Đây là những mảng con chứa các phần tử đã có sẵn thứ tự tăng dần. Việc tìm ra các mảng con như thế có thể giúp ích cho công việc sắp xếp những mảng cha có kích thước rất rất lớn. Hãy viết chương trình tìm mảng con tăng dài nhất trong một mảng cha

INPUT

Một con số n (cho trước giới hạn 1 < n < 103 sinh viên KHÔNG CẦN PHẢI kiểm tra giá trị n) là độ dài của mảng. n số tiếp theo trong input mỗi số có giá trị trong khoảng từ 0 đến 1000 là giá trị các phần tử trong mảng

OUTPUT

Độ dài của mảng con tăng dài nhất

VÍ DỤ

Intput	Output
16 9 2 3 4 5 1 8 5 6 7 2 3 4 5 2 3	4
8 9 2 2 3 4 5 1 8	5

Lời giải:

```
#include <iostream>
using namespace std;
int main()
{
    int a[1000],n,d,t=1,i;
    cin>>n;
    for(int i=0;i<n;i++)
    {
        cin>>a[i];
    }
    i=0;
    while(i<n)
    {
        int j;
        j=i+1;
        d=1;
        while (a[j]>=a[j-1]&&j<n)
        {
            d++;
            j++;
        }
        t=max(d,t);
        i=j;
    }
    cout<<t;
}
```

BT3: **Tần suất**

Bảng tần suất (histogram) là một bảng liệt kê các giá trị và số lần xuất hiện của chúng trong dãy. Lập được bảng tần suất cho phép chúng ta dễ dàng đưa ra nhận xét về các giá trị trong dãy và là bước quan trọng trong các bài toán thống kê.

INPUT

Input là một dãy số nguyên dương, mỗi số được cho trên một hàng, giá trị mỗi số không quá một trăm tỷ. Input sẽ kết thúc bằng một hàng chỉ có số 0.

Giới hạn: input sẽ không chứa qua 10^5 số

OUTPUT

Xuất ra bảng tần suất của các giá trị trong dãy. Mỗi giá trị xuất trên một hàng bao gồm giá trị và số lần xuất hiện cách nhau bởi khoảng trắng. Các giá trị được xuất ra theo thứ tự từ lớn đến bé

VÍ DỤ

Input	Output
2 4 2 4 2 2 1 2 2 4 3 2 4 0	4 4 3 1 2 7 1 1

Lời giải:

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;
int main()
{
    long long a[100000];
    int n,d;
    n=0;
    cin>>a[n];
    while (a[n]!=0)
    {
        n++;
        cin>>a[n];
    }
    sort(a,a+1+n);
    int i=n;
    while (i>0)
    {
        d=1;
        while (a[i]==a[i-1] && i>1)
        {
            d++;
            i--;
        }
        cout<<a[i]<<" "<<d<<endl;
        i--;
    }
}
```

Ghi chú: hàm sort dùng để sắp xếp mảng theo thứ tự tăng dần

BT4: **Búp bê gỗ** (làm thêm)

Công ty đồ chơi X nhập khẩu n con búp bê gỗ. Các con búp bê được đánh số từ 1 tới n trong đó con búp bê thứ i là một hộp rỗng có kích thước là một số nguyên a_i. Người ta có thể lồng con búp bê thứ i vào trong con búp bê thứ j nếu con búp bê thứ j đang rỗng và $a_i+k \le a_j$, với k là một số nguyên dương cho trước. Bằng cách lồng các con búp bê vào nhau theo cách như vậy, công ty X chỉ cần tìm chỗ đặt những con búp bê ngoài cùng (những con búp bê không nằm trong bất kỳ con búp bê nào khác) vào kho.

Yêu cầu: Hãy giúp công ty X lồng các con búp bê vào nhau sao cho tổng kích thước các con búp bê ngoài cùng là nhỏ nhất.

Dữ liệu vào: gồm 2 dòng

- Dòng 1 chứa hai số nguyên dương $n \le 10^5$; $k \le 10^9$ cách nhau một khoảng trắng.
- Dòng 2 chứa n số nguyên dương $a_1, a_2, \ldots, a_n$ ($a_i \le 10^9$), mỗi số cách nhau một khoảng trắng.

Dữ liệu ra:

- Là một số nguyên duy nhất là tổng kích thước các con búp bê ngoài cùng theo phương án tìm được.

Ví dụ:

Input:

8 2

8 4 2 1 1 3 5 9

Output:

18

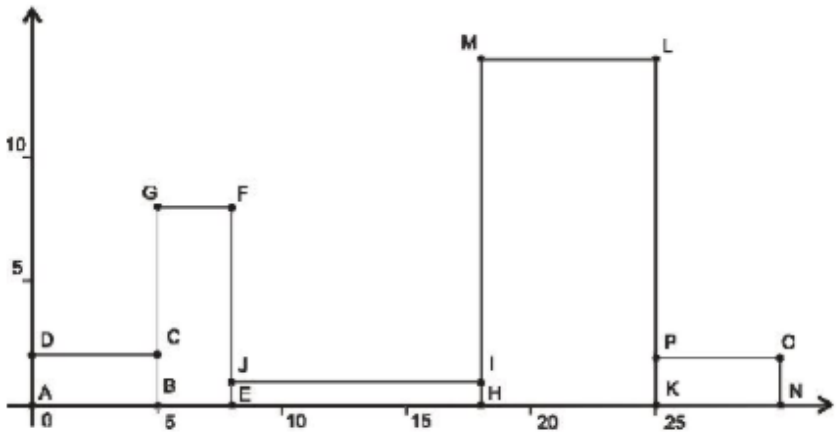
Nguồn: http://ntucoder.net/Problem/Details/48

```
#include <bits/stdc++.h>
using namespace std;
const int N = 202020;
int n, m, k, a[N];

int main()
{
    cin >> n >> k;
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    sort(a + 1, a + 1 + n, greater<int>()); // hàm sắp xếp giảm dần
    long long j = 1, sum = a[1];
    for (int i = 2; i <= n; i++)
        if (a[i] + k <= a[j])
            j++;
        else
            sum += a[i];
    cout << sum;
    return 0;
}
```

BT5: **Hình chữ nhật** (làm thêm)

Cho n hình chữ nhật đánh số từ 1 đến n, hình chữ nhật thứ i có chiều dài là a_i và chiều rộng là b_i. Các hình chữ nhật này được đặt tiếp xúc với trục Ox và nằm kề nhau từ trái qua phải theo thứ tự chỉ số. Mỗi hình chữ nhật có thể tiếp xúc với trục Ox theo cạnh bất kỳ (xem hình). Cần tính độ dài lớn nhất của đường gấp phía trên (xem hình).



Dữ liệu nhập:
- Dòng đầu tiên ghi số hình chữ nhật n. (1 ≤ n ≤ 100)
- Trong n dòng tiếp theo, dòng thứ i ghi hai số a_i và b_i, độ dài cạnh của hình chữ nhật thứ i. (1 ≤ a_i, b_i ≤ 1000).

Dữ liệu xuất:
- Là độ dài lớn nhất tìm được.

Ví dụ

input
5
2 5
3 8
1 10
7 14
2 5

output
68

Giải thích
Cách xếp mà thu được chiều dài lớn nhất là hình trên. Cạnh phía trên gồm các đoạn DC, CG, GF, FJ, JI, IM, ML, LP, và PO.
Độ dài của đoạn này là 5 + 6 + 3 + 7 + 10 + 13 + 7 + 12 + 5 = 68
Nguồn: http://ntucoder.net/Problem/Details/75

Lời giải:

```
#include <bits/stdc++.h>

using namespace std;
int n;
int A[105],B[105];
int F[105][5];

int main()
{
    cin>>n;
    for(int i=1; i<=n ; i++) cin>>A[i]>>B[i];
    F[1][0]=A[1]; F[1][1]=B[1];
    for(int i=2; i<=n ; i++)
    {
        F[i][0]=max(F[i-1][0]+abs(B[i-1]-B[i]),F[i-1][1]+abs(A[i-1]-B[i])) + A[i];
        F[i][1]=max(F[i-1][0]+abs(B[i-1]-A[i]),F[i-1][1]+abs(A[i-1]-A[i])) + B[i];
    }
    cout<<max(F[n][0],F[n][1]);
    return 0;
}
```

Phần II: MẢNG 2 CHIỀU

BT1: **Mảng đối xứng**

Viết chương trình kiểm tra một ma trận có phải là ma trận đối xứng qua trục dọc hay không

INPUT

Dòng đầu tiên chứa hai số r và c là số dòng và số cột của mảng, mỗi số không quá 1000.

r dòng tiếp theo, mỗi dòng chứa c số nguyên. Đây là các phần tử trong mảng 2 chiều

OUTPUT

Xuất ra TRUE nếu có và FALSE nếu không

VÍ DỤ

Input	Output
2 5 12042 30563 24105 30563 12042 25363 30304 18436 30304 25363	TRUE

5 5 28913 26846 85 19649 8524 7970 31692 11666 2213 6747 22640 18740 30793 9860 15977 7970 31692 11666 2213 6747 28913 26846 85 19649 8524	FALSE

Lời giải:

```
#include <iostream>
using namespace std;
const int N = 1000;
bool check(int a[][N], int m, int n)
{
    for (int i=0;i<m;i++)
    {
        int j=0;
        while (j<n/2)
        {
            if (a[i][j]!=a[i][n-j-1]) return( false);
            else j++;
        }
    }
    return (true);
}
int main()
{
    int a[N][N],m,n;
    cin >> m >> n;
    for (int i=0; i < m; i++)
    for (int j=0; j < n; j++)
        cin >> a[i][j];
    if (check(a,m,n)) cout<<"TRUE";
    else cout<<"FALSE";
}
```

BT2: **Đội trưởng**

Một nhóm người đang lên kế hoạch để làm việc nhưng gặp một trở ngại là vẫn chưa tìm ra ai là đội trưởng để phụ trách một mảng công việc nhỏ. Vì thực lực của mỗi người là như nhau và một người có thể làm ở nhiều công việc nhỏ cùng một lúc. Tổng số thành viên của nhóm vừa hay là một số chính

phương nên đã thống nhất vị trí làm việc và công việc sẽ làm thành một ma trận **nxn**. Một trong số họ nảy ra ý tưởng là chọn những người có chiều cao lớn 8 người đứng xung quanh mình để làm nhóm trưởng. Họ đã nhờ một bạn là sinh viên trường UIT lập trình một thuật toán để tìm ra những người đội trưởng đó.

INPUT

Dòng đầu tiên là số nguyên dương n.

n dòng tiếp theo, mỗi dòng chứa n số nguyên là chiều cao của mỗi người.

OUTPUT

Vị trí của nhóm trưởng gồm có chỉ số hàng và cột được cách nhau bởi một khoảng trắng và mỗi vị trí là một hàng

VÍ DỤ

Input	Output
5 28913 26846 85 19649 8524 7970 31692 11666 2213 6747 22640 18740 30793 9860 15977 24997 18366 18071 38766 20104 18223 27172 2671 101 2170	1 1 3 3
3 1 1 1 1 7 1 1 1 1	1 1

Lời giải:

```
#include <iostream>
using namespace std;
int main()
{
    int a[1000][1000],b[1000][1000],m,n,i,j;
    long long sum;
    cin>>n>>m;
    for (i=0;i<n;i++)
        for (j=0;j<m;j++)
            cin>>a[i][j];
    for (i=1;i<n-1;i++)
        for (j=1;j<m-1;j++)
        {
            if (a[i][j]>a[i][j-1]&&a[i][j]>a[i-1][j]
                &&a[i][j]>a[i+1][j]&&a[i][j]>a[i][j+1]
                &&a[i][j]>a[i-1][j-1]&&a[i][j]>a[i-1][j+1]
                &&a[i][j]>a[i+1][j-1]&&a[i][j]>a[i+1][j+1])
            cout<<i<<" "<<j<<endl;
        }
}
```

BT3: **Gián ăn**

Trên một lưới ô vuông hình chữ nhật MxN, tại mỗi ô (i; j) của lưới ta đặt một lượng thức ăn dành cho gián, tương ứng với số nguyên dương a_{ij} . Từ ô hiện tại, gián có thể bò đến một trong 2 ô khác để ăn hết lượng thức ăn đặt ở ô đó là: ô kề cạnh bên phải và ô kề cạnh bên dưới. Ban đầu chú gián ở vị trí ô (1;1) và giả thiết ô này không chứa thức ăn.

Yêu cầu:

Hãy xác định lộ trình trong lưới của gián sao cho từ ô xuất phát đi đến ô cuối cùng (M; N), gián có thể ăn được một lượng thức ăn nhiều nhất.

Dữ liệu vào:

Dòng đầu ghi hai số nguyên dương M và N (0<M,N £ 100). M dòng tiếp theo, mỗi dòng thứ i, ghi N số nguyên dương a_{ij} (0<= a_{ij} <=100; i = 1..M; j = 1..N)

Dữ liệu ra:

Đưa ra màn hình một số nguyên, là lượng thức ăn lớn nhất mà gián có thể ăn được.

Ví dụ:

Input	Output
4 5 0 5 1 3 4 6 1 9 1 5 1 1 1 4 1 1 3 4 6 3	30
4 5 0 1 2 3 1 0 2 3 4 5 6 1 2 3 1 3 2 4 1 5	21

Lời giải:

```
#include <iostream>
using namespace std;
int main()
{
    int m,n,i,j;
    int a[101][101];
    cin>>m>>n;
    for (i=0;i<=m;i++) {
        a[i][0]=0;
    }
    for (i=0;i<=n;i++){
        a[0][i]=0;
    }
    for (i=1;i<=m;i++)
        for (j=1;j<=n;j++)
        {
            cin>>a[i][j];
            a[i][j]+=max(a[i][j-1],a[i-1][j]);
        }
    cout<<a[m][n];
}
```

BT4: **Đường đi có tổng lớn nhất** (làm thêm)

Cho một bảng A kích thước m x n (1 <= m, n <= 100), trên đó ghi các số nguyên aij (|aij| <= 100). Một người xuất phát tại ô nào đó của cột 1, cần sang cột n (tại ô nào cũng được).

Quy tắc đi: Từ ô (i, j) chỉ được quyền sang một trong 3 ô (i, j + 1); (i - 1, j + 1); (i + 1, j + 1)

Input

Dòng 1: Ghi hai số m, n là số hàng và số cột của bảng.

M dòng tiếp theo, dòng thứ i ghi đủ n số trên hàng i của bảng theo đúng thứ tự từ trái qua phải

Output

Gồm 1 dòng duy nhất ghi tổng lớn nhất tìm được

Example

Input:

5 7
9 -2 6 2 1 3 4
0 -1 6 7 1 3 3
8 -2 8 2 5 3 2
1 -1 6 2 1 6 1
7 -2 6 2 1 3 7

Output:
41
Nguồn: https://vn.spoj.com/problems/QBMAX/

Lời giải:

```
#include <iostream>
using namespace std;
int main()
{
    int m,n,i,j;
    long long a[102][101];
    long long qbmax;
    cin>>m>>n;
    for (i=1;i<=m;i++)
        for (j=1;j<=n;j++)
            cin>>a[i][j];
    for (i=1;i<=n;i++)
    {
        a[0][i]=-1000000000;
        a[m+1][i]=-100000000;
    /*gán cho biến kết quả thật bé để thực hiện công việc tìm đường đi
    ko bị ảnh hưởng bởi những giá trị 2 bên rìa*/
    }
    for (j=2;j<=n;j++)
        for (i=1;i<=m;i++)
            a[i][j]=a[i][j]+max(a[i][j-1],max(a[i-1][j-1],a[i+1][j-1]));
    qbmax=-100000000;
    //gán cho biến kết quả thật bé để thực hiện việc tìm max
    for (i=1;i<=m;i++)
        qbmax=max(qbmax,a[i][n]);
    cout<<qbmax;
}
```

2. Chuỗi:

BT1: Tìm Tiếng

Viết chương trình nhập chuỗi ký tự. Nếu chuỗi nhập vào là chuỗi rỗng, chương trình in ra thông báo “Chuoi rong.”. Ngược lại, chương trình đếm đếm số lần xuất hiện của từng tiếng trong chuỗi.

VÍ DỤ:

Input	Output
	Chuoi rong.
Hoc sinh hoc sinh hoc	Hoc: 1 sinh: 2 hoc: 2

Lời giải:

```
#include <iostream>

using namespace std;
int check(int i, int xsize, string x )
{
	for (int j=i;j<=xsize;j++)
		if (x[j]!=' ') return 1;
	return 0;
}
int main()
{
	string s[100],x,y;
	int cnt=0;
	getline(cin,x,'\n');
	if (x=="") cout<<"Chuoi rong.";
	else{
		int xsize=x.size()-1;
		for (int i=0;i<=xsize;i++){
			if (x[i]!=' ') s[cnt]+=x[i];
			else if (x[i]==' ' && i==0) continue;
			else if (x[i-1]!=' ' && check(i,xsize,x) ) cnt++;
		}
		int A[cnt+1][2]={0};
```

```
        for (int i=0;i<=cnt;i++){
            if (A[i][1]==0) {
                for (int j=i; j<=cnt;j++)
                    if (s[i]==s[j]) {A[i][0]++; A[j][1]=1;}
                cout<<s[i]<<": "<<A[i][0]<<endl;
                }
            }
        }
    return 0;
}
```

BT2: Tìm Tên

Viết chương trình nhập chuỗi ký tự. Nếu chuỗi nhập vào là chuỗi rỗng, chương trình in ra thông báo "Chuoi rong.". Ngược lại, chương trình tìm tên riêng trong chuỗi, biết rằng chuỗi tên riêng là chuỗi có ký tự đầu mỗi tiếng được viết hoa và tên riêng là chuỗi có từ 2 tiếng trở lên.

VÍ DỤ:

Input	Output
	Chuoi rong.
Ten toi co 3 tieng, Ngo Van An. Ten ban 2 tieng, Nguyen An.	Ngo Van An Nguyen An

Lời giải:

```
#include <iostream>

using namespace std;

int main()
{
    string x;
    getline(cin,x,'\n');
    if (x==="") cout<<"Chuoi rong.";
    else{
    string s[100];
    int xsize=x.size(), cnt=0;
```

```
        for (int i=0;i<xsize;i++){
                if (x[i]!=' '&& x[i]!='.'&& x[i]!=',') s[cnt]+=x[i];
                else if (x[i]==' ' && x[i+1]==' ' && i+1<xsize) continue;
                else cnt++;
        }
        int A[cnt+1][2]={0};
        for (int i=0;i<=cnt;i++)
                if (s[i][0]<=90 && s[i][0]>=65) A[i][0]=1;
        for (int i=0;i<cnt;i++){
                if (A[i][0]+A[i+1][0]==2) A[i][1]=1;
        }
    for (int i=0;i<=cnt;i++){
        if (A[i][1]==1){
                cout<<s[i]<<" ";
                for (int j=i+1;j<=cnt;j++){
                        if (A[j][0]==1) {cout<<s[j]<<" "; A[j][1]=0;}
                        else {i=j; cout<<endl; break;}
                                }
                        }
                }
        }
        return 0;

}
```

3. Cấu trúc:

Câu 1: Để chuẩn bị cho Hội khỏe Phù Đổng, nhà trường tổ chức cuộc thi điền kinh chạy 100m, mọi học sinh đều có quyền đăng kí thi. Mỗi người dự thi có một phiếu đăng kí gồm các thông tin: Họ và tên, Nam/Nữ (1 – Nam, 0 – Nữ), Lớp, Thời gian chạy (giây). Sau khi chạy tới đích, thời gian chạy sẽ được ghi vào phiếu đăng kí và các phiếu này được nhập vào máy tính để xử lí. Các lớp có từ 3 học sinh nam (nữ) tham dự trở lên được tính thành tích đồng đội nam (nữ). Anh/chị hãy thực hiện các công việc sau:

a. Định nghĩa (Khai báo) cấu trúc Học sinh (gồm hovaten, gioitinh, lop, thoigianchay).
b. Viết hàm tính tổng số học sinh nam dự thi.
c. Viết hàm xuất ra danh sách các học sinh nam về nhất, bao gồm Họ và tên, Lớp.
d. Viết hàm kiểm tra danh sách có từ 3 học sinh nữ tham dự trở lên không.

Bài giải:

```
a. struct hocsinh
   {
       char hoten[250];
```

```
        int gioitinh;
        char lop[8];
        int thoigianchay;
    };
b. int tonghocsinhnam(hocsinh a[],int n)
    {
        int d=0;
        for (int i=0; i<n; i++)
        {
            if (a[i].gioitinh==1) {d++;}
        }
        return d;
    }
c. void namvenhat(hocsinh a[],int n)
    {
        int m=1000000;
        for (int i=0; i<n; i++)
        {
            if (a[i].gioitinh==1){m=min(m,a[i].thoigianchay);}
        }
        for (int i=0; i<n; i++)
        {
            if (a[i].thoigianchay==m && a[i].gioitinh==1)
    {cout<<a[i].hoten<<" "<<a[i].lop<<endl;}
        }
    }
d. bool kiemtranu(hocsinh a[],int n)
    {
        int d=0;
        for (int i=0; i<n; i++)
        {
            if (a[i].gioitinh==0) {d++;}
        }
        if (d>=3) return 1;
        return 0;
    }
```

Câu 2: Cho đa giác lồi $A_1A_2A_3...A_n$ với các đỉnh $A_i(x_i,y_i)$ có tọa độ nguyên.
Viết hàm thực hiện các công việc sau:
a. Định nghĩa (Khai báo) cấu trúc Point (gồm x, y).
b. Viết hàm tính chu vi đa giác.
c. Viết hàm tính diện tích đa giác.

Bài giải:

```
a. struct point
```

```
    {
        Int x,y;
    };
b. double chuvi(point a[], int n)
    {
        a[n+1]=a[1];
        n++;
        double d,s=0;
        for (int i=1; i<n; i++)
        {
            d=sqrt((a[i].x-a[i-1].x)*(a[i].x-a[i-1].x)+(a[i].y-a[i-
    1].y)*(a[i].y-a[i-1].y));
            s=s+d;
        }
        return s;
    }

c. double dientich(point a[], int n)
    {
        a[n+1]=a[1];
        n++;
        double s=0;
        for (int i=1; i<n; i++)
        {
              s=s+(a[i].x-a[i-1].x)*(a[i].y+a[i-1].y);
        }
        s=abs(s/2);
        return s;
    }
```

Câu 3: *Trích từ Câu 4 (2.5 điểm) trong Đề thi cuối kì I NMLT năm 2013-2014.*

a. Định nghĩa (Khai báo) cấu trúc Sinh viên (gồm MSSV, họ tên, năm sinh, điểm trung bình). *(0.5 điểm)*

b. Viết hàm tìm điểm trung bình lớn nhất trong danh sách mảng một chiều có n sinh viên. *(1 điểm)*

c. Viết hàm xuất lên màn hình danh sách những sinh viên có điểm trung bình < 5.0 trong mảng một chiều danh sách có n sinh viên. Các thông tin phải xuất là MSSV, họ tên, năm sinh, điểm trung bình. *(1 điểm)*

Bài giải:

```
a. struct sinhvien
    {
        char mssv[8];
        char hoten[250];
        int namsinh;
        double diemtrungbinh;
    };
```

```
b. double diemtblonnhat(sinhvien a, int n)
   {
         double m=0;
         for (int i=0; i<n; i++)
               {
               m=max(m,a[i].diemtrungbinh);
               }
         return m;
   }
c. void xuat(sinhvien a, int n)
   {
         for (int i=0; i<n; i++)
         {
               if      (a[i].diemtrungbinh<5.0)       {cout<<a[i].mssv<<"
         "<<a[i].hoten<<"                              "<<a[i].namsinh<<"
         "<<a[i].diemtrungbinh<<endl;}
         }
   }
```

4. Con trỏ:**Hoán vị**

```
#include <string>
using namespace std;
int Hoan_Vi_1(int x,int y)
{
   int temp;
   temp= x;x=y;y=temp;
   return 0;
}
int Hoan_Vi_2(int *x,int y)
{
   int temp;
   temp= *x;*x=y;y=temp;
   return 0;
}
int main()
{
 int a = 5,b = 6;

 Hoan_Vi_1(a,b);
 cout<<a<<" "<<b<<endl;
 Hoan_Vi_2(&a,b);
 cout<<a<<" "<<b;
 return 0;
}
```

Ghi kết quả xuất ra sau khi chạy chương trình

Kết quả:

5 6

6 6

5. Đệ quy:

BT1: **Biểu diễn nhị phân**

Cho một số nguyên dương n. Hãy in ra biểu diễn nhị phân của n.

Dữ liệu nhập:

- Là số nguyên dương n ($0 \le n \le 10^9$)

Dữ liệu xuất:

- Biểu diễn nhị phân của n.

Ví dụ

Input: 29

Output: 11101

Bài giải:

```
#include <iostream>
using namespace std;
int NhiPhan(int n)
{
    if (n==1) return 1;
    cout<<NhiPhan(n/2);
    return n%2;
}
int main()
{
    int n;
    cout<<"Nhap vao so nguyen duong n"<<endl;
    cin>>n;
    cout<<”Ket qua là: “<<NhiPhan(n);
    return 0;
}
```

BT2:**Lãi suất ngân hàng**

Viết một hàm đệ quy để tính tổng số tiền nhận được sau khi gửi ngân hàng n tháng với lãi suất 0.015/tháng.

Lưu ý số tiền lãi nhận được sẽ được cộng vào cả gốc để tính lãi cho tháng sau (lãi kép).

Bài giải:

```
#include <iostream>
using namespace std;
```

```
float TienNhan(int Thang ,int BanDau )
{
   if (Thang==0) return BanDau;          //nếu chưa gửi tháng nào thì tiền bạn vẫn không tăng
   return TienNhan(Thang-1, BanDau)*1.15;          //tiền tháng này bằng tiền tháng trước *1.15
}
int main()
{
   int Thang,BanDau;
   cout<<"Nhap vao so thang gui can gui tien"<<endl;
   cin>>Thang;
   cout<<"Nhap vao so tien can gui"<<endl;
   cin>>BanDau;
   cout<<"So tien nhan duoc la "<<TienNhan(Thang,BanDau)<<"d";
   return 0;
}
```

BT3: **Tạo mảng**

Bình đang làm bài tập về mảng, đề yêu cầu tạo ra một mảng tăng dần hoặc giảm dần. Nhưng tạo mảng tầm thường thì quá đơn giản với Bình, Bình là một sinh viên giỏi và đam mê lập trình cơ. Vậy nên Bình muốn nghĩ ra một phương pháp tạo mảng với các điều kiện như sau:

- Cho trước số đầu và số cuối của mảng
- Các số trong mảng phải tăng hoặc giảm đều nhau.
- Bài làm không cần dùng if-else.

Để làm được như vậy Bình đã nghĩ tới phương pháp như sau:

1. Gán giá trị đầu và cuối.
2. Giá trị của số ở giữa mảng sẽ được gán bằng trung bình cộng của số đầu và số cuối. Trường hợp số lượng phần tử của mảng là số chẵn thì xem như mảng có 02 số ở giữa và giá trị 02 số này bằng nhau.
3. Chia mảng làm 02 phần bằng nhau, một phần từ đầu đến giữa và một phần từ giữa đến cuối. Lặp lại bước 1 với từng phần nhỏ này trong mảng

Sau khi nghĩ ra một giải thuật quá trời thông minh và đỉnh đạc như trên thì Bình chợt phát hiện ra một sự thật đau lòng là Bình code không nổi. Bạn hãy code giúp Bình đi.

INPUT

03 con số nguyên lần lượt là: Số phần tử của mảng (không quá 1 triệu), giá trị phần tử đầu, giá trị phần tử cuối

OUTPUT

Xuất ra các số trong mảng sau cùng, mỗi số trên một hàng

VÍ DỤ

Input	Output
10 6 16	6 7.25 8.5 9.75 11 11 12.25 13.5 14.75 16
9 16 6	16 14.75 13.5 12.25 11 9.75 8.5 7.25 6

Lời giải:

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP

```
#include <iostream>
using namespace std;
void chia(double a[],int x,int y){
    if (y-x>=2){
        if ((y-x)%2==1){
            a[(y+x-1)/2]=(a[x]+a[y])/2;
            a[(y+x+1)/2]=(a[x]+a[y])/2;
            chia(a,x,(y+x-1)/2);
            chia(a,(y+x+1)/2,y);
        }
        if ((y-x)%2==0){
            a[(y+x)/2]=(a[x]+a[y])/2;
            chia(a,x,(y+x)/2);
            chia(a,(y+x)/2,y);
        }
    }
}
int main(){
    double a[100000],s;
    int n,d,t=1,i;
    cin>>n;
    cin>>a[0]>>a[n-1];
    s=(a[0]+a[n-1])/2;
    chia(a,0,n-1);
    for (int i=0;i<n;i++){
        cout<<a[i]<<endl;
        }
}
```